

CLAI - Judging-round Q&A prep

Team drill sheet. Answers are plain first, then scholar jargon if the panel presses. End every answer with the soundbite.

Audience note: Judges only know what you said in the presentation. They will not know file paths, env vars, or internal ticket names unless you put them on a slide. Questions below are phrased like post-pitch follow-ups; answers keep the precise numbers ready if they dig.

22 prompts | Complexity 6 / Critical 7 / Depth 9

Five soundbites to memorize

1. We're a harness, not a model - tools + memory + budgets + traces + bench.
2. Two planes: explore like an engineer; bound context like a harness.
3. 81-task suite with offline oracle, live dashboard, and same-model CLAI vs pi vs Codex races.
4. Honest gap: verify designed; soft-complete today; bench checks are real.
5. If it isn't in the append-only JSONL, it didn't happen.

Complexity

How the system is put together - seams, planes, concurrency, budgets.

Q1 - Identity

What is CLAI, in one sentence a judge can repeat?

PLAIN (say this first)

CLAI is a terminal coding agent harness: one binary that wraps any LLM with live tools, memory, context budgets, traces, and a built-in bench - not a chatbot glued to a single model.

SCHOLAR JARGON (if they push)

It is a model-agnostic agentic coding harness: an orchestration layer over the Vercel AI SDK that externalizes control (tools), persistence (tiered memory), epistemic bound (context compaction), and auditability (append-only traces).

SOUNDBITE

"We're a harness, not a model. The model is swappable; the control loop is the product."

Q2 - Two planes

You mentioned architecture / layers - how does CLAI actually split the work?

PLAIN (say this first)

Tool plane finds and changes code (grep, LSP, bash, edit). Harness plane remembers, budgets context, verifies, and records. Tools explore; the harness keeps the agent honest and replayable.

SCHOLAR JARGON (if they push)

Separation of concerns between an operational tool plane (live repo instrumentation) and a control/harness plane (memory provenance, token budgets, verification contract, append-only telemetry). Discovery stays tool-native; durable cognition stays harness-native.

SOUNDBITE

"Explore like an engineer. Remember and bound context like a harness."

Q3 - Seams

What are the main modules, and why split them that way?

PLAIN (say this first)

adapter = agent loop + providers; tools = repo actions; agents = task subagents; shell = background jobs; sandbox = isolation + approvals; memory = SQLite/JSON store; context = assemble + compact; trace = JSONL audit; bench = eval suite + dashboard; ui = terminal shell. Split so we can swap providers or harden sandbox without rewriting the loop.

SCHOLAR JARGON (if they push)

Modular seams with narrow interfaces: the adapter is the policy loop; tools are capability endpoints; agents are bounded subordinate controllers; sandbox is a mediated execution boundary; memory/context implement a dual store; trace is an immutable event log; bench is an in-tree evaluation harness.

SOUNDBITE

"Seams so we can swap providers or harden sandbox without rewriting the loop."

Q4 - Parallelism

Does the agent do things in parallel, or is it one tool at a time?

PLAIN (say this first)

Three places: (1) multiple tool calls in one model step run together, (2) a parallel tool batches up to 6 read-only jobs, (3) multiple task subagents launched in one step run together. Compare races also fan out carefully so sides don't thrash the API.

SCHOLAR JARGON (if they push)

Intra-step tool concurrency, a bounded parallel read aggregator (≤ 6), and multi-agent fork-join on task calls. Evaluation uses partitioned side-parallelism to keep comparative runs load-fair under shared rate limits.

SOUNDBITE

"Parallel where it's safe (reads/tasks), bounded where it's expensive (API + bash)."

Q5 - Subagents

You talked about subagents / task agents - what can they do, and what can't they?

PLAIN (say this first)

explore is read-only by default; general adds bash. They summarize back to the parent in a short report. They cannot edit/write files, cannot spawn another task, and don't inherit background shell jobs - so child context stays small and capability stays limited.

SCHOLAR JARGON (if they push)

Hierarchical delegation with capability attenuation: subordinate controllers inherit a reduced tool set, hard step budgets, and a lossy summarization channel to the parent - preventing recursive spawn and job leakage.

SOUNDBITE

"Subagents explore and report; the parent decides and edits."

Q6 - Smart context

How do you keep the model from blowing past the context window?

PLAIN (say this first)

Always on in chat/run: clean the user prompt, compact mid-turn history when near the window, fold parallel task results, and if you still overflow - compact again and retry. Status shows things like 'prompt cleaned' / 'compacted context' and context %.

SCHOLAR JARGON (if they push)

A deterministic context-management pipeline: prompt hygiene -> soft/hard watermark compaction (~70%/90% of window; soft also capped ~45k tokens) -> multi-task result folding -> overflow compact-and-retry. Context is a scarce resource under explicit budget policy, not an unbounded chat log.

SOUNDBITE

"We don't pray the window holds - we enforce budgets mid-loop."

Critical

Pressure tests - honesty gaps, tradeoffs, failure modes, judge traps.

Q7 - Honesty gap

When you say the agent is 'done,' how do you know it actually worked?

PLAIN (say this first)

Design goal: done means PASS / FAIL / BLOCKED with evidence. Today the live agent loop is still soft completion - the model stops when it thinks it's done. The bench suite's check scripts are real external verification; wiring that same verify contract fully into the live loop is still in progress.

SCHOLAR JARGON (if they push)

Verification-contract / implementation asymmetry: the normative completion ontology exists, but the control loop still uses soft termination. Bench oracles provide exogenous verification; endogenous verify is still landing.

SOUNDBITE

"We're honest: verify is designed and proven on the bench; soft-complete in the live loop today."

Q8 - Sandbox

What if isolation / sandboxing can't install on someone's machine?

PLAIN (say this first)

Install still succeeds. Sandbox falls back to a structured stub; memory can fall back to JSON if SQLite natives are missing. Help, demos, and offline bench still work. Heavy natives are optional so the CLI stays usable.

SCHOLAR JARGON (if they push)

Graceful degradation via optional native deps + stub mediation: when the sandbox runtime is unavailable, we degrade isolation fidelity rather than fail closed on install. Memory similarly degrades from SQLite to JSON.

SOUNDBITE

"Shipability over brittle native coupling - degrade isolation, don't brick the CLI."

Q9 - Why not peers

Why build CLAI instead of wrapping Claude Code / Codex / OpenCode?

PLAIN (say this first)

Peers optimize for product surface - many models, IDE plugins, polished UX. CLAI optimizes for judge-grade traces, memory discipline, reproducible in-tree benches, and same-model harness races. Different job to be done.

SCHOLAR JARGON (if they push)

Different objective function: CLAI is evaluation- and audit-centric (append-only provenance, budgeted context, in-tree harness comparisons). Product agents maximize surface area; we maximize controllability, measurability, and replay.

SOUNDBITE

"We're not cloning a product OS - we're building a measurable coding harness."

Q10 - Compare fairness

Your compare / race results - do those prove you have a better model?

PLAIN (say this first)

No. We race CLAI vs pi vs Codex on the same DeepSeek-flash model. That measures harness quality under a fixed model - tool loop, parallelism, stalls, scoring - not 'our LLM is smarter.'

SCHOLAR JARGON (if they push)

Same-model, cross-harness evaluation: holding the foundation model constant isolates the orchestration substrate as the independent variable. Model quality is intentionally controlled for.

SOUNDBITE

"Same model, different harness - we're scoring the control loop."

Q11 - Trust / injection

Repos can contain malicious text. How do you think about trust?

PLAIN (say this first)

Repo text and tool output can be hostile. We label untrusted data vs trusted memory, scrub secrets from the process env, and gate network egress, destructive actions, and out-of-repo paths behind approvals (unless you explicitly auto-approve for demos).

SCHOLAR JARGON (if they push)

Dual-channel trust model: untrusted data vs trusted memory demarcation, plus mediated execution (deny-by-default approvals) and secret scrubbing at the process boundary - least-privilege / confused-deputy mitigation for agent tooling.

SOUNDBITE

"Treat the repo as untrusted input; treat harness memory as cited and stale-aware."

Q12 - Eval size

How serious is your evaluation - is this a toy demo suite?

PLAIN (say this first)

No. We ship an 81-task in-tree bench with isolated workspaces and oracle check scripts, plus offline mode, a live dashboard, and same-model races against peer harnesses. Started from a smaller seed set; grew into a Terminal-Bench / DeepSWE-style corpus.

SCHOLAR JARGON (if they push)

In-tree microbenchmark corpus with exogenous oracles and multi-harness comparative runs - evaluation is a first-class product surface, not a slide-deck afterthought.

SOUNDBITE

"81 tasks, offline oracle, live dashboard, peer harness races - eval is part of the product."

Q13 - Approvals

Is this safe by default when it can run shell and hit the network?

PLAIN (say this first)

Default is deny until approved for egress, destructive ops, and leaving the workspace. Auto-approve exists for demos and benches - that's an explicit trust trade, not the default posture.

SCHOLAR JARGON (if they push)

Policy-as-guardrail with human-in-the-loop mediation: deny-by-default over privileged side effects, with an explicit auto-approve escape hatch for non-interactive evaluation.

SOUNDBITE

"Safe default, explicit override for headless eval."

Depth

Mechanism-level answers - numbers, defaults, why this design.

Q14 - Providers

Are you locked to one model vendor?

PLAIN (say this first)

No. Providers live in a registry. Default is Groq; you switch provider or model via env without rewriting the agent loop. Eight first-class providers today (Groq, OpenRouter, Cerebras, OpenAI, Anthropic, Gemini, Vercel AI Gateway, DeepSeek).

SCHOLAR JARGON (if they push)

Provider indirection via a registry atop the Vercel AI SDK - the control loop is provider-invariant; only credentials and model-id defaults are pluggable. Breadth is intentional rather than encyclopedic.

SOUNDBITE

"Add a provider in the registry; don't fork the agent loop."

Q15 - Compaction numbers

Any concrete numbers for when you compact context?

PLAIN (say this first)

Soft compact around min(45,000 tokens, 70% of window). Hard around 90% of window. Keep last ~10 turns by default. Most providers ~128k window; Anthropic ~200k. Tunable via env knobs.

SCHOLAR JARGON (if they push)

Dual-watermark token budgeting: soft threshold triggers proactive history compression; hard watermark is the overflow boundary before compact-and-retry. Retention keeps a trailing window for local coherence.

SOUNDBITE

"Soft at ~45k/70%, hard at 90%, keep ~10 turns."

Q16 - Tool caps

Why show the model a truncated tool output instead of everything?

PLAIN (say this first)

The model sees a capped preview (e.g. read ~8KB head+tail, bash ~4KB, grep <=100 matches). Full output still lands in the trace. Truncation protects the context budget; the audit log keeps completeness for debugging and review.

SCHOLAR JARGON (if they push)

Asymmetric observability: lossy projection into the model context window vs near-complete persistence in the append-only event log - token economy without sacrificing forensic replay.

SOUNDBITE

"Small to the model, full to the trace."

Q17 - Memory tiers

Why not just stuff everything into a bigger system prompt?

PLAIN (say this first)

We use typed memory tiers (task, convention, evidence, preference) with TTLs and provenance. Session-local stuff stays session-local. You can cite, invalidate, supersede. Backend is SQLite with JSON fallback - queryable, not just chat history.

SCHOLAR JARGON (if they push)

A typed, provenance-bearing memory ontology with TTL classes and staleness/ablation gates - separating durable institutional knowledge from ephemeral working memory, instead of undifferentiated prompt accretion.

SOUNDBITE

"Memory is a store with citations and expiry - not a bigger system prompt."

Q18 - Traces

What do you mean by judge-grade / replayable traces?

PLAIN (say this first)

Append-only JSONL event logs per run: start/end, model steps, tools, approvals, errors, context stages. If it can't be replayed from the log, we treat it as if it didn't happen. A glass pane can tail assembly live.

SCHOLAR JARGON (if they push)

Immutable event sourcing for agent runs: an append-only telemetry ledger enabling post-hoc reconstruction, attribution, and evaluation. Replayability is the epistemic standard for claimed work.

SOUNDBITE

"If it isn't in the JSONL, it didn't happen."

Q19 - Bench suite

Walk me through how your bench / eval actually works.

PLAIN (say this first)

81 fixture workspaces with a task spec + check script oracle. Offline mode can apply a known solution and still run checks. Live mode runs the agent. Dashboard streams progress; compare scripts race peer harnesses on the same model.

SCHOLAR JARGON (if they push)

In-tree microbenchmark corpus with exogenous oracles, isolated workspaces, and a live observation surface - plus same-model multi-harness races for relative orchestration performance under shared rate limits.

SOUNDBITE

"One command: offline truth, live agent, live dashboard, peer harness races."

Q20 - Loop budgets

What stops the agent from looping forever or hanging on bash?

PLAIN (say this first)

Parent step budget default 12; subagent default 10; bash timeout 60s; provider retries on flaky 429/5xx with backoff. Auth/schema errors fail fast - we don't retry a bad key.

SCHOLAR JARGON (if they push)

Finite-horizon control with bounded retries: a step budget bounds agent depth; temporal timeouts bound tool liveliness; exponential backoff with jitter handles transient provider faults while auth/schema faults fail closed.

SOUNDBITE

"12 parent steps, 10 subagent, 60s bash, retry the flaky HTTP - not the bad key."

Q21 - Intake / LSP

How does CLAI learn a codebase before editing?

PLAIN (say this first)

Live tools: repo intake for a map, grep/glob/read for content, LSP for diagnostics/symbols. Memory is for durable facts you already earned - you don't search memory instead of opening files.

SCHOLAR JARGON (if they push)

Online program comprehension via instrumented tools (structural intake + lexical search + language-server semantics), rejecting memory-as-index substitution. Memory augments; it does not replace repository ground truth.

SOUNDBITE

"Read the code. Memory cites what we've already proven."

Q22 - UI / headless

Is this only a pretty terminal UI, or does it run in CI / headless too?

PLAIN (say this first)

Terminal ADE shell (header, activity, context strip, footer) speaks through one event bus. Same events can drive plain headless printing when there's no TTY - one API, two renderers.

SCHOLAR JARGON (if they push)

Presentation/control separation: a single event bus fans out to an Ink ADE view or a headless sink, so interactive and non-interactive execution share the same observability contract.

SOUNDBITE

"One bus - pretty in the terminal, plain in CI."

How to drill (2 minutes)

- One person reads the question out loud - no peeking at answers.
- Speaker answers in Plain English only (30-45 seconds).
- If the 'judge' says 'go deeper,' escalate to Scholar jargon.
- Close with the Soundbite - one sentence they can write down.
- Mark any miss; re-run those five before the real panel.

Tier cheat sheet

Complexity: seams, two planes, parallelism caps, subagent limits, smart context.

Critical: soft-verify honesty, stub sandbox, why not peers, same-model compare fairness, trust, eval seriousness, approvals.

Depth: 8 providers, 45k/70%/90% compact, tool truncation vs traces, memory TTLs, 81 tasks, step budgets 12/10, headless bus.